Benchmark de algoritmos de ordenamiento

Tarea Extraclase #2 — Algoritmos y Estructuras de Datos I (CE 1103)

José Cabezas Josephs / 2025110797

José Miguel Alfaro Chan / 2022437485

Escuela de Ingeniería en Computadores

Instituto Tecnológico de Costa Rica

GitHub: `https://github.com/Tokitou14/Tarea-II.git`

II Semestre 2026

Resumen

Este informe estudia cinco algoritmos de ordenamiento (Selection, Bubble, Insertion, Shell y Merge) implementados en Java sin optimizaciones. Primero se plantea una predicción teórica y luego se verifica que las implementaciones ordenen correctamente distintos tipos y tamaños de arreglos. Además, se comparará el comportamiento esperado con los tiempos de ejecución medidos, siguiendo la metodología del enunciado [1]. **Ampliación del estudio.** A la versión inicial con cinco algoritmos se añadieron Quicksort y Radix Sort, la verificación de los siete algoritmos y la medición experimental de sus tiempos. Las nuevas secciones presentan los resultados obtenidos en una sola computadora del grupo.

1. Parte 1: Predicción

La siguiente tabla contiene una predicción hecha antes de ejecutar los algoritmos. Por eso, representa lo que esperamos que ocurra según la forma en que está escrito cada programa; no son resultados medidos. La predicción se conserva sin cambios para poder compararla después con los datos del experimento. Se consideran las implementaciones solicitadas, sin optimizaciones adicionales.

Notación utilizada

Sea n la cantidad de elementos del arreglo. La notación $O(\cdot)$ describe cómo aumenta el trabajo del algoritmo cuando aumenta la cantidad de elementos:

- $O(n)$ **(lineal)**: el trabajo aumenta aproximadamente en la misma proporción que el tamaño del arreglo. Si se duplica n, se espera cerca del doble de trabajo.
- $O(n \log n)$ **(linealítmica)**: el arreglo se divide en partes más pequeñas y se procesa cada nivel. Al duplicar n, el trabajo aumenta un poco más del doble.
- $O(n^{3/2})$ **(intermedia)**: es un crecimiento entre $O(n \log n)$ y $O(n^2)$. Al duplicar n, el trabajo aumenta aproximadamente 2,8 veces.

- $O(n^2)$ **(cuadrática)**: el trabajo crece proporcionalmente al cuadrado del tamaño. Al duplicar n, se espera cerca de cuatro veces el trabajo.

El *mejor caso* es el tipo de entrada que requiere menos trabajo; el *caso promedio* representa el comportamiento esperado con entradas comunes, como arreglos aleatorios; y el *peor caso* es una entrada que obliga al algoritmo a realizar más trabajo. Estas categorías describen el comportamiento esperado y no significan que todos los arreglos de una misma categoría produzcan exactamente el mismo tiempo.

Algoritmo	**Mejor**	**Promedio**	**Peor**	**Justificación**
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	En cada posición busca el menor elemento entre todos los que quedan. Hace esa búsqueda aunque el arreglo ya esté ordenado.
Bubble	$O(n^2)$	$O(n^2)$	$O(n^2)$	Esta versión siempre realiza todas las pasadas y todas las comparaciones, incluso si el arreglo ya está ordenado.
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	Si el arreglo está ordenado, cada elemento necesita muy pocas comparaciones. Si está en orden inverso, cada elemento debe desplazarse por toda la parte anterior.
Shell	$O(n \log n)$	$O(n^{3/2})$	$O(n^2)$	Compara elementos separados por saltos que se van reduciendo. Los saltos iniciales pueden acercar elementos a su posición, aunque el trabajo depende de la entrada y de la secuencia de saltos.
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Divide el arreglo repetidamente en mitades y luego mezcla los elementos de cada nivel. Cada nivel procesa, en conjunto, los n elementos.

Tipo de entrada que puede provocar el mejor y el peor caso

Algoritmo	**Mejor caso esperado**	**Peor caso esperado**
Selection	Cualquier orden inicial: la cantidad de comparaciones y de intercambios ejecutados es la misma en esta implementación.	Cualquier orden inicial: la cantidad de comparaciones y de intercambios ejecutados es la misma en esta implementación.
Bubble	Ascendente: no necesita realizar intercambios.	Descendente: se realizan muchos intercambios, uno por cada par que debe corregirse.
Insertion	Ascendente: cada elemento ya está en la posición adecuada y no necesita desplazamientos.	Descendente: cada elemento debe desplazarse a través de todos los elementos anteriores.
Shell	Ascendente: los elementos ya están ordenados para cada salto y casi no se necesitan desplazamientos.	Una entrada especialmente desordenada que provoque muchos desplazamientos durante varias pasadas.
Merge	Ascendente: al mezclar, normalmente se pueden agotar antes las mitades y hacer menos comparaciones.	Mitades cuyos valores estén intercalados: se requieren muchas comparaciones para decidir el orden de los elementos.

En esta parte se hacen predicciones razonadas a partir de los ciclos, comparaciones, desplazamientos y divisiones de cada implementación. En particular, Selection Sort ejecuta el intercambio en cada pasada incluso cuando el mínimo ya está en su posición; por eso, el orden inicial no cambia la cantidad de intercambios que ejecuta este código. La tabla no pretende afirmar que una entrada concreta siempre produzca el menor o mayor tiempo medido: los tiempos también dependen del equipo y de las condiciones de ejecución.

Complemento teórico: Quicksort y Radix Sort

Nota de trazabilidad: las dos tablas anteriores se conservan exactamente como se redactaron originalmente, antes de las mediciones. El análisis teórico que sigue se añadió con posterioridad para documentar los otros dos algoritmos; por tanto, no se presenta como una predicción previa al experimento.

Algoritmo	**Mejor**	**Promedio**	**Peor**	**Justificación**
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Con pivote fijo (último elemento), el costo depende de la distribución de tamaños de las particiones. Si son muy desiguales, la recursión alcanza profundidad lineal.
Radix	$O(n)^*$	$O(d(n+b))$	$O(d(n+b))$	Recorre el arreglo para conocer el máximo y luego realiza una pasada estable por dígito decimal; $b = 10$ y d depende del valor máximo.

[*] Si el máximo es cero, la implementación hace solo la búsqueda inicial del máximo. Para un número fijo $d \geq 1$ de dígitos, el costo es $\Theta(d(n+b))$.

Algoritmo	**Mejor caso esperado**	**Peor caso esperado**
Quicksort	Arreglo cuyos pivotes dividen repetidamente los elementos en dos grupos de tamaños semejantes.	Arreglos ascendentes o descendentes con pivote final; también numerosos valores repetidos pueden crear particiones desequilibradas.
Radix	Valores con pocos dígitos (y, para esta versión, todos ceros es el caso mínimo).	Valores de más dígitos a igual n; con el mismo d, cambiar el orden inicial afecta poco el número de pasadas.

2. Parte 2: Implementación con apoyo de IA

Las cinco implementaciones se escribieron en Java (IntelliJ IDEA), sin optimizaciones, y cada una se valida contra `Arrays.sort` de la biblioteca estándar [1]. Para cada algoritmo se documenta lo mismo y en el mismo orden: prompt, código, explicación de la IA, operaciones, contribución de la IA, correcciones y ajustes, y verificación.

2.1. Selection Sort

2.1.1. Prompt y código

Prompt. [2] Se adjuntó el PDF del enunciado de la tarea a la conversación con la IA.

```
CONTEXTO: Soy estudiante de Ingeniería en Computadores y estoy haciendo un
    benchmark de algoritmos de ordenamiento en Java, te pase el pdf de la tarea
    para que sepas de lo que vamos a trabajar.

TAREA: Genera la implementación del algoritmo indicado abajo.

ALGORITMO: Selection Sort
- Versión clásica: en cada iteración busca el mínimo del subarreglo restante y lo
    intercambia con la posición actual.

REQUISITOS:
1. Lenguaje: Java. Una clase con un método estático: public static void sort(int[]
    arr)
2. Ordena el arreglo en el lugar (in-place), de forma ascendente.
3. SIN optimizaciones: implementación estándar, sin banderas de salida anticipada
    ni trucos extra.
4. No uses Arrays.sort, Collections.sort ni ninguna función de ordenamiento de la
    biblioteca.
5. Solo la biblioteca estándar de Java, sin dependencias externas.
6. Código claro, con nombres de variables descriptivos y comentarios breves en espa
    ñol.

FORMATO DE SALIDA:
a) El código completo en un solo bloque.
b) Una explicación breve (máximo 5 líneas) de cómo funciona.
c) La cantidad de comparaciones e intercambios que hace, en términos de n.
```

Código.

```java
public class SelectionSort {

    public static void sort(int[] arr) {

        // Recorre cada posición del arreglo, excepto la última
        for (int i = 0; i < arr.length - 1; i++) {

            // Supone que el elemento actual es el mínimo
            int indiceMinimo = i;

            // Busca el elemento mínimo del subarreglo restante
            for (int j = i + 1; j < arr.length; j++) {

                if (arr[j] < arr[indiceMinimo]) {
                    indiceMinimo = j;
                }
            }

            // Intercambia el mínimo encontrado con la posición actual
            int temporal = arr[i];
            arr[i] = arr[indiceMinimo];
            arr[indiceMinimo] = temporal;
        }
    }
}
```

2.1.2. Respuesta de la IA

Explicación.

1. Se recorre el arreglo desde la primera posición hasta la penúltima.
2. En cada iteración se busca el índice del elemento mínimo del subarreglo restante.
3. Una vez encontrado, se intercambia con el elemento de la posición actual.
4. Al finalizar cada iteración, la posición actual queda ordenada definitivamente.
5. El proceso continúa hasta que todo el arreglo queda ordenado de forma ascendente.

Operaciones que realiza (según la IA). Sea n la cantidad de elementos del arreglo.

Operación	Mejor caso	Caso promedio	Peor caso
Comparaciones	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
Intercambios	$n-1$	$n-1$	$n-1$

Las comparaciones salen de $(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$. Se hace un intercambio al final de cada iteración del ciclo externo.

Contribución de la IA. La IA generó el código de `SelectionSort`, la explicación de su funcionamiento y el conteo de comparaciones e intercambios. Las restricciones del algoritmo (versión clásica, sin optimizaciones, in-place, sin funciones de ordenamiento de la biblioteca) las definimos nosotros en el prompt.

2.1.3. Correcciones y ajustes

- **Cambios al código generado:** ninguno. El código se usó tal como lo entregó la IA.
- **Revisión manual del código:**
  - El ciclo externo llega hasta `arr.length - 1` porque el último elemento queda en su lugar cuando los demás ya están ordenados.
  - El ciclo interno empieza en `i + 1`, así que solo revisa el subarreglo restante.
  - El intercambio usa una variable temporal y es correcto.
  - No usa `Arrays.sort` ni otra función de ordenamiento de la biblioteca, y ordena en el mismo arreglo.
  - No tiene optimizaciones: no hay salida anticipada y el intercambio se hace siempre, incluso cuando el mínimo ya está en la posición actual.
- **Ajuste al prompt:** en la versión final se indicó a la IA que se adjuntaba el PDF del enunciado, para que conociera el contexto de la tarea.
- **Observación:** la IA indica $n - 1$ intercambios en todos los casos, porque el intercambio se ejecuta aunque el mínimo ya esté en su lugar. Este dato se contrasta con la Parte 1 en la comparación de la Parte 4.

2.2. Bubble Sort

2.2.1. Prompt y código

Prompt. [2]

```
CONTEXTO: Soy estudiante de Ingeniería en Computadores y estoy haciendo un
    benchmark de algoritmos de ordenamiento en Java.

TAREA: Genera la implementación del algoritmo indicado abajo.

ALGORITMO: Bubble Sort
- Versión clásica con dos ciclos anidados que comparan elementos adyacentes e
    intercambian si están en el orden incorrecto.
- SIN bandera de salida anticipada (no debe detenerse aunque el arreglo ya esté
    ordenado) y SIN reducir el rango de la pasada interna en cada vuelta.

REQUISITOS:
1. Lenguaje: Java. Una clase con un método estático: public static void sort(int[]
    arr)
2. Ordena el arreglo en el lugar (in-place), de forma ascendente.
3. SIN optimizaciones: implementación estándar, sin banderas de salida anticipada
    ni trucos extra.
4. No uses Arrays.sort, Collections.sort ni ninguna función de ordenamiento de la
    biblioteca.
5. Solo la biblioteca estándar de Java, sin dependencias externas.
```

```
6. Código claro, con nombres de variables descriptivos y comentarios breves en espa
    ñol.

FORMATO DE SALIDA:
a) El código completo en un solo bloque.
b) Una explicación breve (máximo 5 líneas) de cómo funciona.
c) La cantidad de comparaciones e intercambios que hace, en términos de n.
```

Código.

```java
public class BubbleSort {

    public static void sort(int[] arr) {

        // Recorre el arreglo completo en cada pasada
        for (int i = 0; i < arr.length - 1; i++) {

            // Compara todos los pares de elementos adyacentes
            for (int j = 0; j < arr.length - 1; j++) {

                // Intercambia si los elementos están en orden incorrecto
                if (arr[j] > arr[j + 1]) {
                    int temporal = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temporal;
                }
            }
        }
    }
}
```

2.2.2. Respuesta de la IA

Explicación.

1. El ciclo externo controla las pasadas sobre el arreglo.
2. El ciclo interno compara todos los pares de elementos adyacentes.
3. Si el elemento actual es mayor que el siguiente, ambos se intercambian.
4. Cada pasada mueve los elementos mayores hacia el final del arreglo.
5. Al terminar las pasadas, el arreglo queda ordenado de forma ascendente.

Operaciones que realiza (según la IA). Sea n la cantidad de elementos del arreglo. El ciclo externo hace $n-1$ pasadas y el interno hace $n-1$ comparaciones en cada una.

Operación	Mejor caso	Caso promedio	Peor caso
Comparaciones	$(n-1)^2$	$(n-1)^2$	$(n-1)^2$
Intercambios	0	$\approx \frac{n(n-1)}{4}$	$\frac{n(n-1)}{2}$

El mejor caso de intercambios corresponde a un arreglo ordenado ascendentemente, el promedio a uno aleatorio y el peor a uno ordenado descendentemente.

Contribución de la IA. La IA generó el código de `BubbleSort`, la explicación de su funcionamiento y el conteo de comparaciones e intercambios. Las restricciones (sin bandera de salida anticipada, sin reducir el rango de la pasada interna, in-place, sin funciones de ordenamiento de la biblioteca) las definimos nosotros en el prompt.

2.2.3. Correcciones y ajustes

- **Cambios al código generado:** ninguno. El código se usó tal como lo entregó la IA.
- **Revisión manual del código:**
  - El ciclo externo hace $n-1$ pasadas y el interno recorre todos los pares adyacentes con `j <arr.length - 1`, así que `arr[j + 1]` nunca se sale del arreglo.
  - La comparación es estricta (`>`), por lo que los elementos iguales no se intercambian.
  - El intercambio usa una variable temporal y es correcto.
  - No tiene bandera de salida anticipada ni reduce el rango del ciclo interno, como pide el enunciado al exigir una implementación sin optimizaciones.
  - No usa `Arrays.sort` ni otra función de ordenamiento de la biblioteca, y ordena en el mismo arreglo.
- **Ajuste al prompt:** las restricciones de no usar bandera de salida anticipada ni reducir el rango se escribieron de forma explícita en el prompt, porque muchas versiones de Bubble Sort incluyen esas optimizaciones y habría que quitarlas a mano.
- **Observación:** como no se reduce el rango del ciclo interno, esta versión hace $(n-1)^2$ comparaciones y no los $\frac{n(n-1)}{2}$ de la versión que sí lo reduce. Sigue siendo $\Theta(n^2)$, pero la constante es mayor y se tendrá en cuenta al interpretar los tiempos y las razones $t(2n)/t(n)$.

2.3. Insertion Sort

2.3.1. Prompt y código

Prompt. [2]

```
CONTEXTO: Soy estudiante de Ingeniería en Computadores y estoy haciendo un
    benchmark de algoritmos de ordenamiento en Java.

TAREA: Genera la implementación del algoritmo indicado abajo.

ALGORITMO: Insertion Sort
- Versión clásica: toma cada elemento y lo desplaza hacia la izquierda hasta su
    posición correcta dentro de la parte ya ordenada.
- Comparación lineal hacia atrás (SIN búsqueda binaria).

REQUISITOS:
1. Lenguaje: Java. Una clase con un método estático: public static void sort(int[]
    arr)
2. Ordena el arreglo en el lugar (in-place), de forma ascendente.
```

```
3. SIN optimizaciones: implementación estándar, sin banderas de salida anticipada
    ni trucos extra.
4. No uses Arrays.sort, Collections.sort ni ninguna función de ordenamiento de la
    biblioteca.
5. Solo la biblioteca estándar de Java, sin dependencias externas.
6. Código claro, con nombres de variables descriptivos y comentarios breves en espa
    ñol.

FORMATO DE SALIDA:
a) El código completo en un solo bloque.
b) Una explicación breve (máximo 5 líneas) de cómo funciona.
c) La cantidad de comparaciones e intercambios que hace, en términos de n.
```

Código.

```java
public class InsertionSort {

    public static void sort(int[] arr) {

        // Recorre el arreglo desde el segundo elemento
        for (int i = 1; i < arr.length; i++) {

            // Guarda el elemento que se va a insertar
            int valorActual = arr[i];

            // Comienza comparando con el elemento anterior
            int j = i - 1;

            // Desplaza hacia la derecha los elementos mayores
            while (j >= 0 && arr[j] > valorActual) {
                arr[j + 1] = arr[j];
                j--;
            }

            // Inserta el elemento en su posición correcta
            arr[j + 1] = valorActual;
        }
    }
}
```

2.3.2. Respuesta de la IA

Explicación.

1. Se recorre el arreglo desde el segundo elemento, considerando que el primero ya está ordenado.
2. Se guarda el elemento actual para insertarlo en su posición correcta.
3. Se compara linealmente hacia atrás con los elementos de la parte ordenada.
4. Los elementos mayores se desplazan una posición hacia la derecha.
5. Finalmente, se inserta el elemento en su posición correspondiente.

Operaciones que realiza (según la IA). Sea n la cantidad de elementos del arreglo. Esta implementación no hace intercambios tradicionales: desplaza elementos hacia la derecha.

Operación	Mejor caso	Caso promedio	Peor caso
Comparaciones	$n-1$	$\approx \frac{n^2}{4}$	$\frac{n(n-1)}{2}$
Desplazamientos	0	$\approx \frac{n(n-1)}{4}$	$\frac{n(n-1)}{2}$

El mejor caso corresponde a un arreglo ordenado ascendentemente, el promedio a uno aleatorio y el peor a uno ordenado descendentemente. La complejidad temporal que indica la IA es $\Theta(n)$ en el mejor caso y $\Theta(n^2)$ en el promedio y en el peor.

Contribución de la IA. La IA generó el código de `InsertionSort`, la explicación de su funcionamiento y el conteo de comparaciones y desplazamientos. Las restricciones (versión clásica, búsqueda lineal hacia atrás sin búsqueda binaria, sin optimizaciones, in-place, sin funciones de ordenamiento de la biblioteca) las definimos nosotros en el prompt.

2.3.3. Correcciones y ajustes

- **Cambios al código generado:** ninguno. El código se usó tal como lo entregó la IA.
- **Revisión manual del código:**
  - El ciclo externo empieza en `i = 1`, porque el primer elemento ya forma una parte ordenada de tamaño 1.
  - La condición del `while` revisa primero `j >= 0`, así que `arr[j]` nunca se sale del arreglo.
  - La comparación es estricta (`>`), por lo que los elementos iguales no se desplazan.
  - Se guarda `valorActual` antes de desplazar, y se inserta al final en `arr[j + 1]`.
  - La búsqueda es lineal hacia atrás (sin búsqueda binaria), no usa `Arrays.sort` ni otra función de la biblioteca, y ordena en el mismo arreglo.
- **Observación:** la IA usa desplazamientos (una asignación por paso) en lugar de intercambios completos. Esto es parte de la versión clásica y se tendrá en cuenta al interpretar los tiempos. Los casos que indica la IA coinciden con la Parte 1: mejor caso lineal con entrada ascendente y peor caso cuadrático con entrada descendente.

2.4. Shell Sort

2.4.1. Prompt y código

Prompt. [2]

```
CONTEXTO: Soy estudiante de Ingeniería en Computadores y estoy haciendo un
    benchmark de algoritmos de ordenamiento en Java.

TAREA: Genera la implementación del algoritmo indicado abajo.

ALGORITMO: Shell Sort
- Es un Insertion Sort con saltos (gaps) decrecientes.
- Usa la secuencia original de Shell: n/2, n/4, ..., 1 (dividiendo el salto entre 2
     en cada vuelta).
```

```
REQUISITOS:
1. Lenguaje: Java. Una clase con un método estático: public static void sort(int[]
    arr)
2. Ordena el arreglo en el lugar (in-place), de forma ascendente.
3. SIN optimizaciones: implementación estándar, sin banderas de salida anticipada
    ni trucos extra.
4. No uses Arrays.sort, Collections.sort ni ninguna función de ordenamiento de la
    biblioteca.
5. Solo la biblioteca estándar de Java, sin dependencias externas.
6. Código claro, con nombres de variables descriptivos y comentarios breves en espa
    ñol.

FORMATO DE SALIDA:
a) El código completo en un solo bloque.
b) Una explicación breve (máximo 5 líneas) de cómo funciona.
c) La cantidad de comparaciones e intercambios que hace, en términos de n.
```

Código.

```java
public class ShellSort {

    public static void sort(int[] arr) {

        // Comienza con un salto igual a la mitad del tamaño del arreglo
        for (int salto = arr.length / 2; salto > 0; salto /= 2) {

            // Recorre el arreglo utilizando el salto actual
            for (int i = salto; i < arr.length; i++) {

                // Guarda el elemento que se va a insertar
                int valorActual = arr[i];

                // Comienza comparando con el elemento anterior según el salto
                int j = i;

                // Desplaza los elementos mayores hacia la derecha según el salto
                while (j >= salto && arr[j - salto] > valorActual) {
                    arr[j] = arr[j - salto];
                    j -= salto;
                }

                // Inserta el elemento en su posición correcta
                arr[j] = valorActual;
            }
        }
    }
}
```

2.4.2. Respuesta de la IA

Explicación.

1. Se inicia con un salto (gap) igual a la mitad del tamaño del arreglo.

2. Para cada salto, se aplica una variante de Insertion Sort que compara elementos separados por esa distancia.

3. Los elementos mayores se desplazan hacia la derecha según el salto actual.

4. Después de cada pasada, el salto se divide entre dos.

5. El algoritmo termina cuando el salto llega a 1, dejando el arreglo ordenado.

Operaciones que realiza (según la IA). Sea n la cantidad de elementos del arreglo. La secuencia de saltos es $\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \ldots, 1$. La IA indica que no existe una fórmula exacta sencilla en función de n para todas las entradas, y que esta implementación no hace intercambios tradicionales sino desplazamientos, como Insertion Sort.

Operación	**Mejor caso**	**Caso promedio**	**Peor caso**
Comparaciones	$O(n \log n)$	depende de la entrada	$O(n^2)$
Desplazamientos	0	depende de la entrada	$O(n^2)$

Contribución de la IA. La IA generó el código de `ShellSort`, la explicación de su funcionamiento y el análisis de comparaciones y desplazamientos. La secuencia de saltos (la original de Shell, dividiendo entre 2) y las demás restricciones las definimos nosotros en el prompt.

2.4.3. Correcciones y ajustes

- **Cambios al código generado:** ninguno. El código se usó tal como lo entregó la IA.
- **Revisión manual del código:**
  - El salto inicial es `arr.length / 2` y se divide entre 2 en cada vuelta hasta llegar a 0, es decir, la secuencia es $\frac{n}{2}, \frac{n}{4}, \ldots, 1$. La última pasada usa salto 1, que es un Insertion Sort completo, y eso garantiza que el arreglo quede ordenado.
  - La condición del `while` revisa primero `j >= salto`, así que `arr[j - salto]` nunca se sale del arreglo.
  - La comparación es estricta (`>`) y se guarda `valorActual` antes de desplazar.
  - Con un arreglo de 0 o 1 elementos el salto inicial es 0, no se ejecuta ningún ciclo y el arreglo queda igual, lo cual es correcto.
  - No usa secuencias alternativas, bandera de salida anticipada ni funciones de ordenamiento de la biblioteca, y ordena en el mismo arreglo.
- **Observación:** la IA no da una complejidad para el caso promedio. En la Parte 1 estimamos $O(n^{3/2})$, y eso se contrastará con las mediciones en la Parte 4.

2.5. Merge Sort

2.5.1. Prompt y código

Prompt. [2]

```
CONTEXTO: Soy estudiante de Ingeniería en Computadores y estoy haciendo un
    benchmark de algoritmos de ordenamiento en Java (IntelliJ IDEA, JDK 17 o
    superior.

TAREA: Genera la implementación del algoritmo indicado abajo.

ALGORITMO: Merge Sort
- Versión clásica top-down y recursiva: divide el arreglo en dos mitades, ordena
    cada una recursivamente y las mezcla.
- Usa un arreglo auxiliar para la mezcla (merge).

REQUISITOS:
1. Lenguaje: Java. Una clase con un método estático: public static void sort(int[]
    arr)
2. Ordena el arreglo de forma ascendente. El resultado debe quedar en arr (se
    permite usar un arreglo auxiliar interno).
3. SIN optimizaciones: implementación estándar, sin cambiar a Insertion Sort en
    subarreglos pequeños ni saltarse la mezcla si ya están ordenados.
4. No uses Arrays.sort, Collections.sort ni ninguna función de ordenamiento de la
    biblioteca.
5. Solo la biblioteca estándar de Java, sin dependencias externas.
6. Código claro, con nombres de variables descriptivos y comentarios breves en espa
    ñol.

FORMATO DE SALIDA:
a) El código completo en un solo bloque.
b) Una explicación breve (máximo 5 líneas) de cómo funciona.
c) La cantidad de comparaciones e intercambios que hace, en términos de n.
```

Código.

```java
public class MergeSort {

    public static void sort(int[] arr) {

        // Crea el arreglo auxiliar para realizar las mezclas
        int[] auxiliar = new int[arr.length];

        // Inicia el ordenamiento recursivo
        mergeSort(arr, auxiliar, 0, arr.length - 1);
    }

    private static void mergeSort(int[] arr, int[] auxiliar, int inicio, int fin) {

        // Caso base: un arreglo de un elemento ya está ordenado
        if (inicio >= fin) {
            return;
        }

        // Calcula el punto medio del arreglo
        int medio = inicio + (fin - inicio) / 2;
```

```
        // Ordena recursivamente la mitad izquierda
        mergeSort(arr, auxiliar, inicio, medio);

        // Ordena recursivamente la mitad derecha
        mergeSort(arr, auxiliar, medio + 1, fin);

        // Mezcla ambas mitades ordenadas
        merge(arr, auxiliar, inicio, medio, fin);
    }

    private static void merge(int[] arr, int[] auxiliar, int inicio, int medio, int
         fin) {

        // Índices para recorrer ambas mitades
        int indiceIzquierdo = inicio;
        int indiceDerecho = medio + 1;
        int indiceAuxiliar = inicio;

        // Compara y copia el menor elemento de ambas mitades
        while (indiceIzquierdo <= medio && indiceDerecho <= fin) {

            if (arr[indiceIzquierdo] <= arr[indiceDerecho]) {
                auxiliar[indiceAuxiliar] = arr[indiceIzquierdo];
                indiceIzquierdo++;
            } else {
                auxiliar[indiceAuxiliar] = arr[indiceDerecho];
                indiceDerecho++;
            }

            indiceAuxiliar++;
        }

        // Copia los elementos restantes de la mitad izquierda
        while (indiceIzquierdo <= medio) {
            auxiliar[indiceAuxiliar] = arr[indiceIzquierdo];
            indiceIzquierdo++;
            indiceAuxiliar++;
        }

        // Copia los elementos restantes de la mitad derecha
        while (indiceDerecho <= fin) {
            auxiliar[indiceAuxiliar] = arr[indiceDerecho];
            indiceDerecho++;
            indiceAuxiliar++;
        }

        // Copia los elementos mezclados al arreglo original
        for (int i = inicio; i <= fin; i++) {
            arr[i] = auxiliar[i];
        }
    }
}
```

2.5.2. Respuesta de la IA

Explicación.

1. Se crea un arreglo auxiliar del mismo tamaño que el arreglo original.
2. Se divide recursivamente el arreglo en dos mitades hasta llegar a subarreglos de un elemento.
3. Se ordenan las mitades recursivamente.
4. Se mezclan ambas mitades comparando sus elementos y colocando primero el menor.
5. Finalmente, los elementos mezclados se copian nuevamente al arreglo original.

Operaciones que realiza (según la IA). Sea n la cantidad de elementos del arreglo. En cada nivel de mezcla se hacen como máximo n comparaciones, y el algoritmo tiene aproximadamente $\log_2 n$ niveles. Merge Sort no hace intercambios tradicionales: copia elementos entre el arreglo original y el auxiliar, aproximadamente n copias por nivel.

Operación	Mejor caso	Caso promedio	Peor caso
Comparaciones	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Copias	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$

La complejidad temporal que indica la IA es $\Theta(n \log n)$ y la espacial es $\Theta(n)$.

Contribución de la IA. La IA generó el código de `MergeSort`, la explicación de su funcionamiento y el análisis de comparaciones y copias. Las restricciones (versión clásica top-down y recursiva, arreglo auxiliar, sin cambiar a Insertion Sort en subarreglos pequeños, sin saltarse la mezcla, sin funciones de ordenamiento de la biblioteca) las definimos nosotros en el prompt.

2.5.3. Correcciones y ajustes

- **Cambios al código generado:** ninguno. El código se usó tal como lo entregó la IA.
- **Revisión manual del código:**
  - El arreglo auxiliar se crea una sola vez en `sort` y se reutiliza en todas las mezclas.
  - El caso base `inicio >= fin` cubre los arreglos de 0 o 1 elementos.
  - El punto medio se calcula como `inicio + (fin - inicio) / 2`, que evita el desbordamiento de enteros.
  - La mezcla usa `<=`, así que los elementos iguales conservan su orden.
  - Al final de cada mezcla se copia el resultado de vuelta a `arr`, por lo que el arreglo ordenado queda en el arreglo original.
  - No tiene optimizaciones: no cambia a Insertion Sort en subarreglos pequeños y mezcla siempre, aunque las mitades ya estén ordenadas. Tampoco usa funciones de ordenamiento de la biblioteca.
- **Observación:** la IA indica $\Theta(n \log n)$ comparaciones en todos los casos, lo que coincide con la Parte 1. La cantidad exacta cambia con la entrada solo por una constante (por ejemplo, la entrada ascendente necesita menos comparaciones por mezcla), y eso se contrastará con las mediciones en la Parte 4. Usa $O(n)$ de memoria adicional por el arreglo auxiliar.

2.6. Quicksort

2.6.1. Prompt y código

Prompt de apoyo utilizado con IA (resumido).

```
CONTEXTO: Benchmark de ordenamiento de CE1103 en Java.
TAREA: Implementar QuickSort clasico en una clase con
public static void sort(int[] arr).
REQUISITOS: pivote fijo en el ultimo elemento (Lomuto),
recursividad, orden ascendente, sin pivote aleatorio,
sin cambiar a Insertion Sort, sin Arrays.sort y con
comentarios en espanol. Explicar costos mejor, promedio
y peor, asi como posibles desbordamientos de pila.
```

El texto anterior resume las indicaciones de esta conversación para documentar la ayuda de IA; no se presenta como una transcripción literal del prompt original.

Código.

```java
public class QuickSort {
    public static void sort(int[] arr) {
        quickSort(arr, 0, arr.length - 1);
    }
    private static void quickSort(int[] arr, int inicio, int fin) {
        if (inicio >= fin) return;
        // Pivote fijo: ultimo elemento del subarreglo
        int posicionPivote = particionar(arr, inicio, fin);
        quickSort(arr, inicio, posicionPivote - 1);
        quickSort(arr, posicionPivote + 1, fin);
    }
    private static int particionar(int[] arr, int inicio, int fin) {
        int pivote = arr[fin];
        int fronteraMenores = inicio - 1;
        for (int j = inicio; j < fin; j++) {
            if (arr[j] <= pivote) {
                fronteraMenores++;
                intercambiar(arr, fronteraMenores, j);
            }
        }
        intercambiar(arr, fronteraMenores + 1, fin);
        return fronteraMenores + 1;
    }
    private static void intercambiar(int[] arr, int a, int b) {
        int temporal = arr[a];
        arr[a] = arr[b];
        arr[b] = temporal;
    }
}
```

2.6.2. Respuesta de la IA

Explicación.

1. Se toma el último elemento como pivote, sin escogerlo al azar.

2. Se recorre el subarreglo y se colocan los elementos menores o iguales al pivote hacia la izquierda.
3. Se coloca el pivote en su posición definitiva y se identifican dos subarreglos.
4. Se aplica el mismo procedimiento recursivamente a cada subarreglo.
5. Cuando un subarreglo tiene cero o un elemento, la recursión termina.

Operación	Mejor	Promedio	Peor
Comparaciones	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$
Intercambios	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Operaciones que realiza (según la IA). El costo de la pila es $O(\log n)$ con particiones equilibradas y $O(n)$ en el peor caso.

Contribución de la IA. La IA generó el código y una primera explicación de su complejidad. La restricción de pivote fijo procede del enunciado del curso; la elección del último elemento y la partición de Lomuto quedaron explícitas en la implementación utilizada.

2.6.3. Correcciones y ajustes

- **Revisión del código:** se comprobó que el pivote sea siempre `arr[fin]`, que no se llame a `Arrays.sort` y que el caso base detenga la recursión.
- **Ajuste metodológico:** el programa de pruebas registra `StackOverflowError` y detiene únicamente la serie afectada; no modifica ni optimiza Quicksort para eliminar el comportamiento que se desea medir.
- **Validación:** la ejecución de la verificación en la computadora del grupo reportó 24 de 24 pruebas correctas para este algoritmo.

2.7. Radix Sort

2.7.1. Prompt y código

Prompt de apoyo utilizado con IA (resumido).

```
CONTEXTO: Benchmark de ordenamiento de CE1103 en Java.
TAREA: Implementar Radix Sort LSD decimal en una clase
con public static void sort(int[] arr).
REQUISITOS: solo enteros no negativos, base 10,
Counting Sort estable por digito desde unidades,
sin Arrays.sort y con comentarios en espanol.
Explicar costo en funcion de n, d y base.
```

El texto anterior es un resumen documentado de la solicitud de IA y no una transcripción literal.

Código.

```java
public class RadixSort {
    public static void sort(int[] arr) {
        if (arr.length < 2) return;
        int maximo = 0;
        for (int valor : arr) {
            if (valor < 0)
                throw new IllegalArgumentException(
                        "Radix solo acepta enteros >= 0");
            if (valor > maximo) maximo = valor;
        }
        int[] auxiliar = new int[arr.length];
        for (long divisor = 1; maximo / divisor > 0;
                divisor *= 10) {
            int[] conteos = new int[10];
            for (int valor : arr)
                conteos[(int) ((valor / divisor) % 10)]++;
            for (int d = 1; d < 10; d++)
                conteos[d] += conteos[d - 1];
            for (int i = arr.length - 1; i >= 0; i--) {
                int digito = (int) ((arr[i] / divisor) % 10);
                auxiliar[--conteos[digito]] = arr[i];
            }
            System.arraycopy(auxiliar, 0, arr, 0, arr.length);
        }
    }
}
```

2.7.2. Respuesta de la IA

Explicación.

1. Se identifica el valor máximo para determinar cuántos dígitos son necesarios.
2. Se empieza por las unidades y se obtiene el dígito correspondiente de cada número.
3. Se cuentan las frecuencias de los diez posibles dígitos y se calculan posiciones acumuladas.
4. Se recorre el arreglo de derecha a izquierda y se distribuyen los números de forma estable en el arreglo auxiliar.
5. Se repite para decenas, centenas y demás posiciones hasta procesar el dígito más significativo.

Operación	Mejor	Promedio	Peor
Búsqueda del máximo	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Conteo y distribución	$O(n)^*$	$\Theta(d(n+b))$	$\Theta(d(n+b))$

Operaciones que realiza (según la IA). Con $b = 10$ fijo y $d \geq 1$, la complejidad temporal es $\Theta(n + d(n + b))$ y la memoria adicional es $\Theta(n + b)$. El caso señalado con asterisco corresponde a un máximo de cero, que no activa pasadas por dígito.

Contribución de la IA. La IA aportó la implementación LSD con conteos acumulados, recorrido inverso estable y el análisis de pasadas por dígito. La restricción de enteros no negativos coincide con los datos exigidos en el enunciado.

2.7.3. Correcciones y ajustes

- **Revisión del código:** se confirmó que el algoritmo recorra las cifras de menor a mayor peso y mantenga la estabilidad en cada pasada.
- **Revisión de límites:** se usa `long` para el divisor y así representar la pasada correspondiente a nueve dígitos sin desbordar el contador de posiciones decimales.
- **Validación:** la ejecución del programa reportó 24 de 24 pruebas correctas y una prueba adicional de nueve dígitos superada.

2.8. Verificación contra la biblioteca estándar

Para comprobar que los cinco algoritmos ordenan correctamente, se creó un programa de verificación independiente. El programa genera arreglos de distintos tamaños y con cuatro tipos de contenido: aleatorio, ascendente, descendente y con muchos valores repetidos. Para cada prueba realiza los siguientes pasos:

1. Genera el arreglo de entrada.
2. Crea dos copias idénticas: una para el algoritmo que se está probando y otra para usarla como referencia.
3. Ordena la primera copia con el algoritmo correspondiente.
4. Ordena la segunda copia con `Arrays.sort`, que sirve como referencia de la biblioteca estándar de Java.
5. Compara ambos resultados con `Arrays.equals`. Si los arreglos son iguales, la prueba se marca como `OK`; de lo contrario, se marca como `FALLO`.

Se probaron seis tamaños (0, 1, 2, 10, 1000 y 5000 elementos) y cuatro tipos de entrada. Por lo tanto, cada algoritmo realizó 24 pruebas. Como se verificaron cinco algoritmos, se realizaron 120 pruebas en total. Se utilizó una semilla fija para generar los datos aleatorios, de modo que las pruebas sean reproducibles y todos los algoritmos reciban las mismas entradas.

2.8.1. Programa principal: `Main.java`

El programa principal únicamente inicia la verificación mediante la llamada a `Verificacion.ejecutar()`. Los dos archivos se encuentran en el paquete `AlgoritmosOrdenamiento`, igual que las clases de los algoritmos.

```
package AlgoritmosOrdenamiento;

public class Main {

    public static void main(String[] args) {
        Verificacion.ejecutar();
    }
}
```

2.8.2. Clase de verificación: `Verificacion.java`

La clase contiene los generadores de entradas, el procedimiento que compara el resultado de cada algoritmo con `Arrays.sort` y el resumen de las pruebas superadas.

```
package AlgoritmosOrdenamiento;

import java.util.Arrays;
import java.util.Random;
import java.util.function.Consumer;

public class Verificacion {

    private static final int VALOR_MAXIMO = 999_999;
    private static final int[] TAMANOS = {0, 1, 2, 10, 1000, 5000};
    private static final String[] TIPOS = {
            "aleatorio", "ascendente", "descendente", "duplicados"
    };

    private static int[] generarAleatorio(int n, Random azar) {
        int[] arreglo = new int[n];
        for (int i = 0; i < n; i++) {
            arreglo[i] = azar.nextInt(VALOR_MAXIMO + 1);
        }
        return arreglo;
    }

    private static int[] generarAscendente(int n, Random azar) {
        int[] arreglo = generarAleatorio(n, azar);
        Arrays.sort(arreglo); // Solo prepara la entrada de prueba
        return arreglo;
    }

    private static int[] generarDescendente(int n, Random azar) {
        int[] arreglo = generarAscendente(n, azar);
        for (int i = 0; i < n / 2; i++) {
            int temporal = arreglo[i];
            arreglo[i] = arreglo[n - 1 - i];
            arreglo[n - 1 - i] = temporal;
        }
        return arreglo;
    }

    private static int[] generarDuplicados(int n, Random azar) {
        int[] arreglo = new int[n];
        for (int i = 0; i < n; i++) {
            arreglo[i] = azar.nextInt(11);
        }
        return arreglo;
    }

    private static int[] generarEntrada(int tipo, int n, Random azar) {
        switch (tipo) {
            case 0: return generarAleatorio(n, azar);
            case 1: return generarAscendente(n, azar);
            case 2: return generarDescendente(n, azar);
            default: return generarDuplicados(n, azar);
        }
```

```
    }

    private static boolean verificar(
            Consumer<int[]> algoritmo, int[] entrada) {

        int[] copiaAlgoritmo = entrada.clone();
        int[] copiaReferencia = entrada.clone();

        algoritmo.accept(copiaAlgoritmo);
        Arrays.sort(copiaReferencia);

        return Arrays.equals(copiaAlgoritmo, copiaReferencia);
    }

    private static int probar(
            String nombre, Consumer<int[]> algoritmo) {

        Random azar = new Random(42);
        int superadas = 0;

        System.out.println("=== " + nombre + " ===");

        for (int n : TAMANOS) {
            StringBuilder linea = new StringBuilder(
                    String.format("n=%-5d", n)
            );

            for (int tipo = 0; tipo < TIPOS.length; tipo++) {
                int[] entrada = generarEntrada(tipo, n, azar);
                boolean correcto = verificar(algoritmo, entrada);

                if (correcto) {
                    superadas++;
                }

                linea.append(" | ")
                     .append(TIPOS[tipo])
                     .append(": ")
                     .append(correcto ? "OK" : "FALLO");
            }

            System.out.println(linea);
        }

        int total = TAMANOS.length * TIPOS.length;
        System.out.println(
                "Resultado: " + superadas + " de " + total
                        + " pruebas superadas\n"
        );

        return superadas;
    }

    public static void ejecutar() {
        String[] nombres = {
                "Selection", "Bubble", "Insertion", "Shell", "Merge"
        };

        int[] superadas = new int[nombres.length];
```

```
        System.out.println("VERIFICACION CONTRA Arrays.sort\n");

        superadas[0] = probar("Selection", SelectionSort::sort);
        superadas[1] = probar("Bubble", BubbleSort::sort);
        superadas[2] = probar("Insertion", InsertionSort::sort);
        superadas[3] = probar("Shell", ShellSort::sort);
        superadas[4] = probar("Merge", MergeSort::sort);

        int totalPorAlgoritmo = TAMANOS.length * TIPOS.length;
        int sumaSuperadas = 0;

        System.out.println("=== RESUMEN ===");

        for (int i = 0; i < nombres.length; i++) {
            sumaSuperadas += superadas[i];

            System.out.println(
                    String.format(
                            "%-10s %2d / %d  %s",
                            nombres[i],
                            superadas[i],
                            totalPorAlgoritmo,
                            superadas[i] == totalPorAlgoritmo
                                    ? "TODAS LAS PRUEBAS PASARON"
                                    : "HAY FALLOS"
                    )
            );
        }

        System.out.println(
                "\nTotal general: " + sumaSuperadas + " de "
                        + (totalPorAlgoritmo * nombres.length)
                        + " pruebas superadas."
        );
    }
}
```

2.8.3. Ejecución y resultados

Para ejecutar la verificación, se abrió `Main.java` en IntelliJ IDEA y se ejecutó su método `main`. La consola mostró que los cinco algoritmos superaron las 24 pruebas correspondientes. El resultado fue de 120 pruebas superadas de 120 realizadas, sin errores.
A continuación se presenta la salida obtenida en la terminal:

```
VERIFICACION CONTRA Arrays.sort

=== Selection ===
n=0     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=2     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=10    | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1000  | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=5000  | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
Resultado: 24 de 24 pruebas superadas

=== Bubble ===
n=0     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
```

```
n=1      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=2      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=10     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=5000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
Resultado: 24 de 24 pruebas superadas

=== Insertion ===
n=0      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=2      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=10     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=5000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
Resultado: 24 de 24 pruebas superadas

=== Shell ===
n=0      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=2      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=10     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=5000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
Resultado: 24 de 24 pruebas superadas

=== Merge ===
n=0      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=2      | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=10     | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=1000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
n=5000   | aleatorio: OK | ascendente: OK | descendente: OK | duplicados: OK
Resultado: 24 de 24 pruebas superadas

=== RESUMEN ===
Selection  24 / 24  TODAS LAS PRUEBAS PASARON
Bubble     24 / 24  TODAS LAS PRUEBAS PASARON
Insertion  24 / 24  TODAS LAS PRUEBAS PASARON
Shell      24 / 24  TODAS LAS PRUEBAS PASARON
Merge      24 / 24  TODAS LAS PRUEBAS PASARON

Total general: 120 de 120 pruebas superadas.

Process finished with exit code 0
```

El código de generación de arreglos ascendentes utiliza `Arrays.sort` únicamente para preparar los datos de prueba. La copia de referencia también se ordena con `Arrays.sort`; ninguno de los cinco algoritmos evaluados utiliza esa función en su implementación.

2.9. Verificación complementaria de los siete algoritmos

La verificación ya documentada en la sección anterior corresponde a la versión inicial del proyecto, con cinco algoritmos y 120 casos correctos. Posteriormente se incorporaron `QuickSort` y `RadixSort` y se amplió el programa `Verificacion.java`. No se reemplaza ni se altera el registro original.

```
VERIFICACION CONTRA Arrays.sort
Selection  24/24 pruebas
Bubble     24/24 pruebas
Insertion  24/24 pruebas
Shell      24/24 pruebas
Merge      24/24 pruebas
Quick      24/24 pruebas
```

```
Radix      24/24 pruebas
Radix 9 digitos: OK
TOTAL: 169/169 pruebas superadas
```

Esta salida corresponde a la verificación ejecutada por el grupo en su computadora. Los siete algoritmos fueron contrastados contra `Arrays.sort`; el experimento extra de Radix cubrió números de nueve dígitos.

3. Parte 3: Experimentos

3.1. Tipos de entrada y tamaños

Se emplearon arreglos de enteros no negativos en el rango $[0, 999\,999]$, con cuatro tipos de entrada: aleatoria, ascendente, descendente y con duplicados (valores de 0 a 10). Selection, Bubble e Insertion se midieron con $n = 1000, 2000, 4000, 8000, 16000$. Shell, Merge, Quicksort y Radix aumentaron el tamaño de la misma forma hasta $n = 1\,024\,000$, salvo los casos de Quicksort en que la recursión alcanzó el límite de pila.
Para la prueba especial de Radix se fijó $n = 100\,000$, base decimal diez y tres rangos de valores: hasta tres, seis y nueve dígitos (valores de 0 a $10^d - 1$).

3.2. Procedimiento de medición

Los datos se generan de manera reproducible mediante una semilla determinada por el tipo de entrada y el tamaño. Cada algoritmo recibe copias del mismo arreglo para un caso dado. La copia, generación y validación se ejecutan fuera del intervalo medido. Se registran cinco ejecuciones temporizadas válidas por combinación de algoritmo, tipo y tamaño; la ejecución previa se descarta como calentamiento de la JVM. La medición se realiza con `System.nanoTime()` y posteriormente se obtiene la mediana con el script de análisis. El CSV individual conserva la duración de cada repetición en nanosegundos.

Comandos empleados.

```
javac -encoding UTF-8 -d out src/*.java src/AlgoritmosOrdenamiento/*.java
java -cp out AlgoritmosOrdenamiento.Main verificar
java -cp out AlgoritmosOrdenamiento.Main benchmark
python scripts/analisis.py resultados/mediciones.csv
```

3.3. Entorno y limitaciones

Las mediciones del informe proceden de una única computadora del grupo. **Equipo empleado:** procesador Intel Core i7-11800H de 11.ª generación (frecuencia base de 2,30 GHz), 16 GB de memoria RAM, sistema operativo Microsoft Windows 11 (edición y compilación no especificadas) y Microsoft Build of OpenJDK 21.0.12 LTS de 64 bits (build 21.0.12+8-LTS). Todos los algoritmos se ejecutaron en ese mismo equipo. Estos datos describen el entorno declarado; el estado de los procesos en segundo plano durante las mediciones no fue registrado.
La ejecución reportada por la terminal registró 224 series válidas, 1 120 mediciones temporizadas exitosas y tres límites de pila de Quicksort: ascendente con $n = 32\,000$, descendente con $n = 32\,000$ y duplicados con $n = 256\,000$. Estos tamaños se documentan como `STACK_OVERFLOW`; se detuvo cada serie afectada y los casos no se sustituyeron por tiempos inventados.

4. Parte 4: Análisis

4.1. Resultados experimentales y medianas

La tabla completa de mediciones individuales se entrega en el archivo `resultados/mediciones.csv`. Las tablas siguientes se generan **a partir del CSV real de la computadora del grupo** mediante `herramientas/generar_tablas_latex.py`; para cada algoritmo incluyen los cuatro tipos de entrada y todos los tamaños que se ejecutaron correctamente.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	0.934	0.175	0.206	0.196
2000	0.825	0.687	0.771	0.722
4000	3.087	2.758	3.322	2.942
8000	12.246	11.452	13.450	11.459
16000	45.617	44.108	52.678	49.972

Tabla 1: Mediana en milisegundos: Selection Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	0.953	0.256	0.738	0.624
2000	2.874	0.981	2.909	2.163
4000	12.498	5.235	11.344	10.035
8000	57.984	16.382	43.307	53.580
16000	276.755	64.780	176.013	263.083

Tabla 2: Mediana en milisegundos: Bubble Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	0.559	0.00120	0.135	0.05120
2000	0.786	0.00190	0.415	0.212
4000	0.988	0.00330	2.201	1.113
8000	3.426	0.00620	8.419	4.655
16000	17.035	0.01190	39.669	19.760

Tabla 3: Mediana en milisegundos: Insertion Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	0.335	0.00580	0.01150	0.02170
2000	0.189	0.01280	0.02440	0.05560
4000	0.379	0.02950	0.05270	0.129
8000	0.680	0.06300	0.113	0.292
16000	1.362	0.140	0.244	0.651
32000	4.086	0.297	0.542	1.369
64000	8.114	1.004	1.260	2.902
128000	16.505	1.911	2.540	6.100
256000	34.584	3.405	5.705	14.078
512000	80.610	6.747	11.738	27.332
1024000	191.373	13.559	27.244	55.366

Tabla 4: Mediana en milisegundos: Shell Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	0.127	0.04850	0.02860	0.03140
2000	0.178	0.05560	0.05280	0.07650
4000	0.421	0.120	0.114	0.187
8000	3.759	0.357	0.235	0.409
16000	1.573	0.550	0.536	1.228
32000	2.974	1.151	1.144	1.806
64000	6.328	3.540	2.589	3.940
128000	13.068	6.280	4.950	7.759
256000	27.724	11.399	12.121	16.750
512000	64.092	23.513	24.030	34.283
1024000	124.947	48.890	50.627	71.713

Tabla 5: Mediana en milisegundos: Merge Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	0.08340	0.339	0.242	0.04490
2000	0.146	1.291	0.929	0.161
4000	0.169	5.441	4.931	0.598
8000	0.376	22.840	16.170	2.125
16000	0.873	92.341	67.587	8.620
32000	1.807	–	–	34.454
64000	3.857	–	–	141.721
128000	7.965	–	–	592.222
256000	17.216	–	–	–
512000	37.186	–	–	–
1024000	74.985	–	–	–

Tabla 6: Mediana en milisegundos: Quick Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	0.265	0.02900	0.04250	0.01070
2000	0.139	0.05740	0.05950	0.02110
4000	0.254	0.115	0.123	0.04930
8000	0.504	0.231	0.244	0.08440
16000	0.635	0.477	0.505	0.168
32000	0.963	1.206	0.992	0.345
64000	2.019	1.978	1.972	0.663
128000	4.693	4.186	4.220	1.350
256000	8.264	10.614	8.410	2.745
512000	17.088	16.901	16.301	5.626
1024000	33.693	38.842	33.991	11.158

Tabla 7: Mediana en milisegundos: Radix Sort según tipo de entrada. Los guiones indican datos no disponibles.

4.2. Razones entre tamaños consecutivos

Para cada tipo de entrada y algoritmo se calcula la razón

$$R(n) = \frac{t(2n)}{t(n)},$$

donde $t(n)$ es la mediana de cinco tiempos de ejecución para el tamaño n. Una razón cercana a dos resulta compatible con crecimiento lineal, una ligeramente superior a dos con $n \log n$ y una próxima a cuatro con crecimiento cuadrático; estos valores son referencias orientativas, no criterios absolutos. Las primeras mediciones pueden mostrar variabilidad por optimización JIT, gestión de memoria y resolución temporal.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	–	–	–	–
2000	0.884	3.925	3.748	3.678
4000	3.742	4.013	4.310	4.073
8000	3.966	4.153	4.048	3.895
16000	3.725	3.852	3.917	4.361

Tabla 8: Razón $t(2n)/t(n)$: Selection Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	–	–	–	–
2000	3.015	3.826	3.940	3.468
4000	4.348	5.338	3.899	4.639
8000	4.640	3.129	3.818	5.339
16000	4.773	3.954	4.064	4.910

Tabla 9: Razón $t(2n)/t(n)$: Bubble Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	–	–	–	–
2000	1.404	1.583	3.071	4.139
4000	1.257	1.737	5.305	5.253
8000	3.470	1.879	3.825	4.181
16000	4.971	1.919	4.712	4.245

Tabla 10: Razón $t(2n)/t(n)$: Insertion Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	–	–	–	–
2000	0.565	2.207	2.122	2.562
4000	2.002	2.305	2.160	2.315
8000	1.795	2.136	2.140	2.272
16000	2.004	2.214	2.164	2.226
32000	2.999	2.128	2.219	2.103
64000	1.986	3.383	2.326	2.120
128000	2.034	1.903	2.017	2.101
256000	2.095	1.782	2.246	2.308
512000	2.331	1.981	2.057	1.942
1024000	2.374	2.010	2.321	2.026

Tabla 11: Razón $t(2n)/t(n)$: Shell Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	–	–	–	–
2000	1.397	1.146	1.846	2.436
4000	2.367	2.158	2.167	2.443
8000	8.936	2.976	2.055	2.188
16000	0.419	1.539	2.279	3.002
32000	1.890	2.094	2.135	1.471
64000	2.128	3.076	2.263	2.182
128000	2.065	1.774	1.912	1.969
256000	2.121	1.815	2.449	2.159
512000	2.312	2.063	1.982	2.047
1024000	1.949	2.079	2.107	2.092

Tabla 12: Razón $t(2n)/t(n)$: Merge Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	–	–	–	–
2000	1.747	3.811	3.834	3.586
4000	1.158	4.215	5.307	3.716
8000	2.231	4.198	3.279	3.552
16000	2.319	4.043	4.180	4.056
32000	2.071	–	–	3.997
64000	2.135	–	–	4.113
128000	2.065	–	–	4.179
256000	2.161	–	–	–
512000	2.160	–	–	–
1024000	2.017	–	–	–

Tabla 13: Razón $t(2n)/t(n)$: Quick Sort según tipo de entrada. Los guiones indican datos no disponibles.

n	Aleatorio	Ascendente	Descendente	Duplicados
1000	–	–	–	–
2000	0.524	1.979	1.400	1.972
4000	1.827	2.000	2.062	2.336
8000	1.986	2.011	1.991	1.712
16000	1.259	2.064	2.069	1.989
32000	1.518	2.530	1.962	2.057
64000	2.095	1.641	1.988	1.920
128000	2.325	2.116	2.140	2.035
256000	1.761	2.535	1.993	2.033
512000	2.068	1.592	1.938	2.050
1024000	1.972	2.298	2.085	1.983

Tabla 14: Razón $t(2n)/t(n)$: Radix Sort según tipo de entrada. Los guiones indican datos no disponibles.

4.3. Gráficas

Se presentan cuatro gráficas: comparación log-log de los siete algoritmos con entrada aleatoria, comparación de cuatro tipos de entrada para Insertion, comparación equivalente para Quicksort y variación de los dígitos en Radix. **Las imágenes deben proceder de la ejecución local** del script de análisis, no de ejemplos de otra computadora.

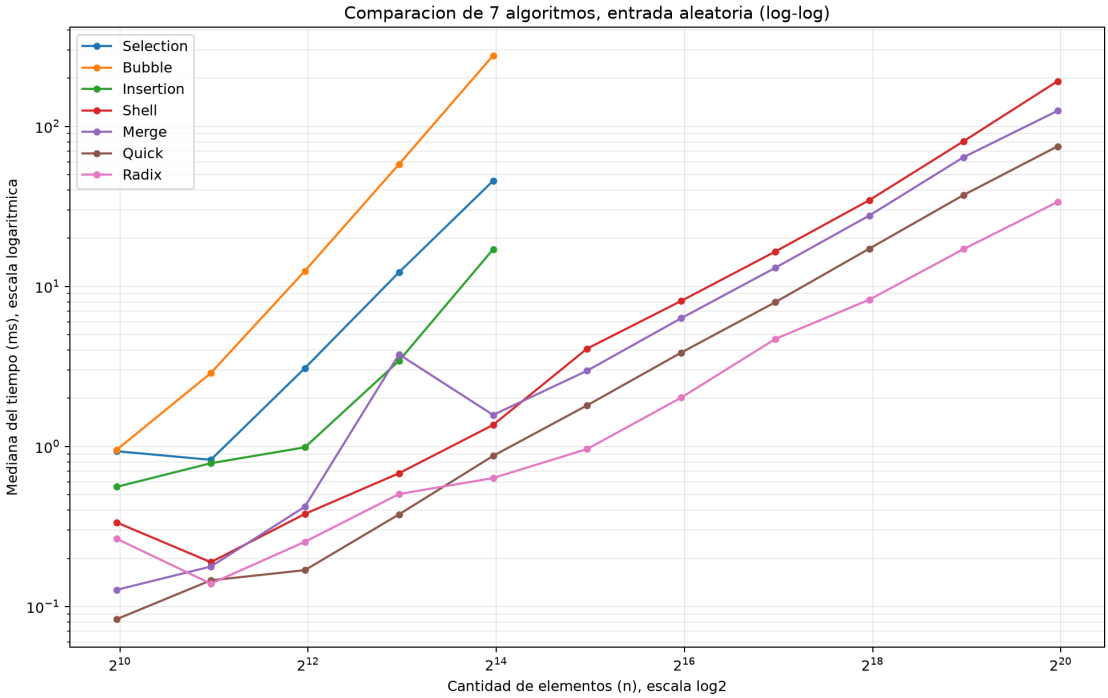


Figura 1: Comparación log-log de los siete algoritmos para arreglos aleatorios.

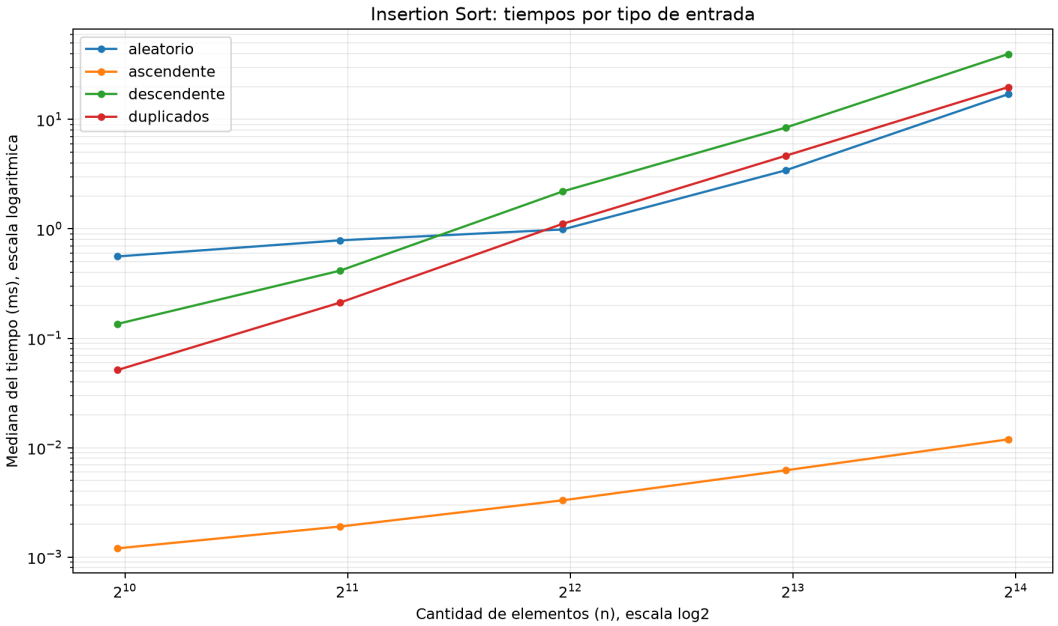


Figura 2: Insertion Sort según el tipo de entrada.

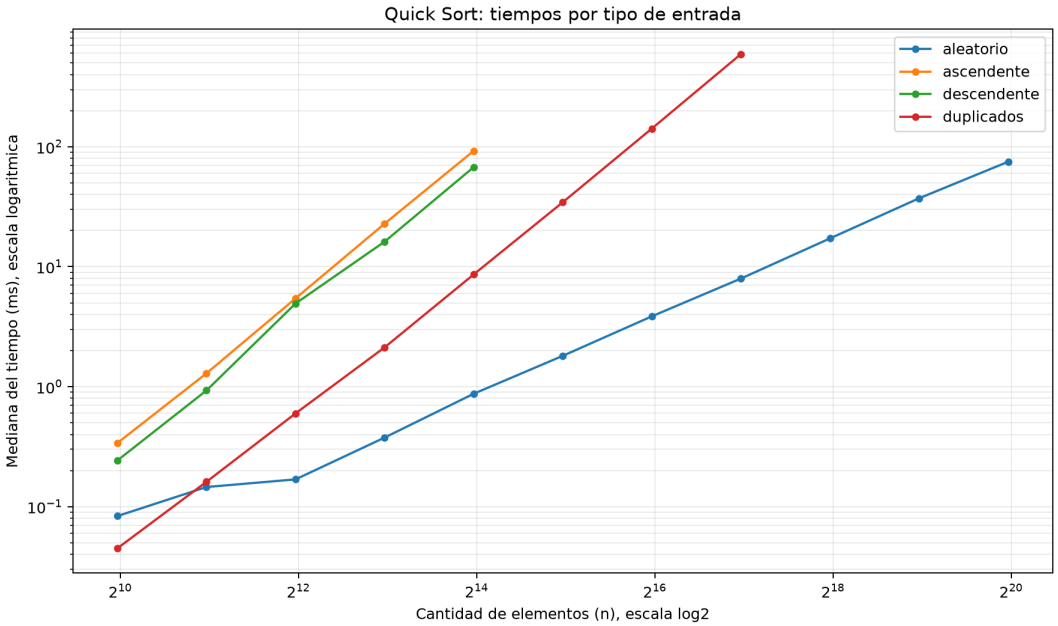


Figura 3: Quicksort según el tipo de entrada; se omiten los tamaños sin medición válida por desbordamiento de pila.

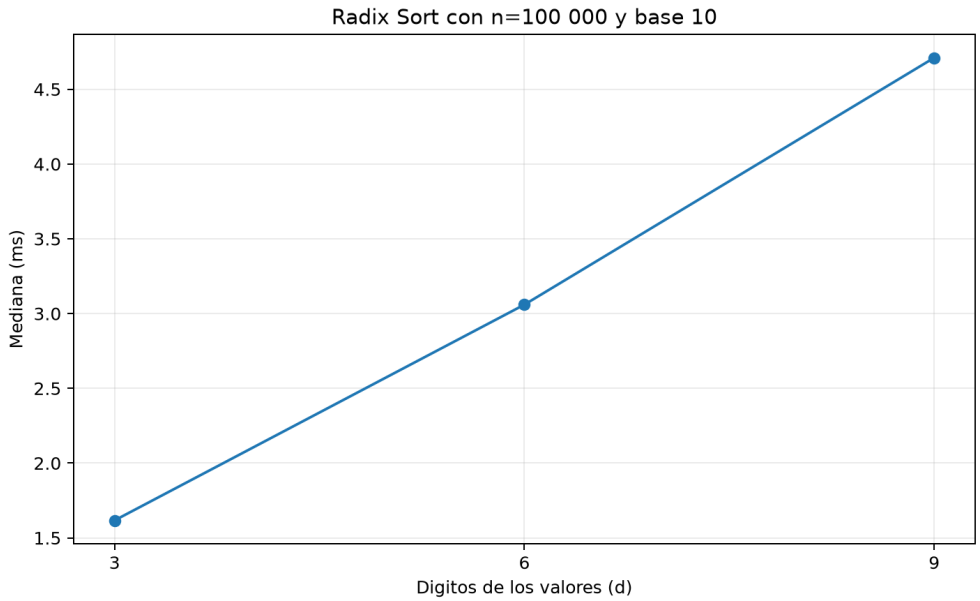


Figura 4: Prueba especial de Radix: tiempo mediano según la cantidad de dígitos.

4.4. Comparación de teoría y experimentos

Los párrafos siguientes contrastan la predicción teórica con las mediciones registradas en el equipo descrito, indicando diferencias entre tipos de entrada y limitaciones del experimento.

Selection Sort. La tabla inicial establece costo cuadrático en los tres casos. El algoritmo recorre el subarreglo restante en cada pasada, independientemente de cómo esté ordenada la entrada. Se espera que sus razones se aproximen a cuatro para tamaños suficientemente grandes; los tiempos individuales pueden experimentar fluctuaciones de medición.
Medición del equipo. Para $n = 16\,000$, la mediana fue 45,617 ms en entrada aleatoria, 44,108 ms en ascendente y 52,678 ms en descendente. La última razón aleatoria fue 3,72, próxima a la referencia cuadrática (4).

Bubble Sort. La implementación del documento no utiliza salida anticipada ni reduce el rango de comparación. Por eso el número de comparaciones sigue siendo $(n-1)^2$ aun para arreglos ascendentes. La cantidad de intercambios sí cambia: es nula para entradas ya ordenadas y elevada en entradas descendentes. Los tiempos pueden variar por esta diferencia aunque la clase asintótica siga siendo cuadrática.
Medición del equipo. Para $n = 16\,000$, la mediana aleatoria fue 276,755 ms y la ascendente 64,780 ms; la última razón aleatoria fue 4,77. El orden inicial modifica los intercambios y los tiempos, aun cuando se conserva el costo cuadrático de las comparaciones.

Insertion Sort. Su mejor caso es lineal cuando los datos ya se encuentran ascendentes, porque no requiere desplazar elementos. En arreglos descendentes realiza aproximadamente $n(n-1)/2$ desplazamientos y presenta un peor caso cuadrático. Los datos aleatorios y duplicados permiten contrastar el efecto del número de desplazamientos en los tiempos.
Medición del equipo. Para $n = 16\,000$, el arreglo ascendente registró 0,01190 ms, el aleatorio 17,035 ms y el descendente 39,669 ms. La última razón aleatoria fue 4,97, mientras que la ascendente fue 1,92. La diferencia concuerda con la cantidad de desplazamientos que exige cada entrada.

Shell Sort. Con saltos $n/2, n/4, \ldots, 1$, la última pasada equivale a una inserción tradicional, aunque las pasadas previas reducen las distancias de movimiento. La predicción inicial del promedio $O(n^{3/2})$ es una aproximación para la secuencia seleccionada, no una fórmula universal exacta. Las razones y las diferencias entre entradas permiten evaluar el efecto de la secuencia de saltos.
Medición del equipo. Para entrada aleatoria y $n = 1\,024\,000$, la mediana fue 191,373 ms y la última razón 2,37. Con la misma cantidad de elementos, la entrada ascendente registró 13,559 ms. Las razones medidas ilustran que una aproximación teórica general no determina exactamente los tiempos.

Merge Sort. Su recurrencia divide el arreglo en dos mitades y mezcla a costo lineal por nivel; el crecimiento esperado es $\Theta(n \log n)$ en todos los casos. Las diferencias entre entradas pueden reflejar el número concreto de comparaciones y el acceso a memoria, pero no cambian la clase asintótica de la versión utilizada.
Medición del equipo. Con entrada aleatoria y $n = 1\,024\,000$ se obtuvo una mediana de 124,947 ms y una razón última de 1,95; en ascendente la mediana fue 48,890 ms. La tendencia de la razón es compatible con crecimiento linealítmico a esta escala.

Quicksort. Con pivote fijo al final, la entrada aleatoria suele producir particiones relativamente equilibradas y crecimiento próximo a $n \log n$. En entradas ascendentes o descendentes se forman particiones extremadamente desequilibradas y el costo puede acercarse a n^2. El desbordamiento de pila registrado en las tres series mencionadas es evidencia del efecto de la profundidad recursiva y debe mostrarse como limitación del experimento.

Medición del equipo. Para entrada aleatoria y $n = 1\,024\,000$, la mediana fue 74,985 ms y la última razón 2,02. Para $n = 16\,000$, las últimas razones ascendentes y descendentes fueron 4,04 y 4,18. Luego, ambas series presentaron desbordamiento de pila en $n = 32\,000$; la de duplicados falló en $n = 256\,000$.

Radix Sort. Con base diez y cantidad fija de dígitos, cada pasada usa un barrido de los n valores y un arreglo de frecuencias de tamaño diez. Se espera crecimiento aproximadamente lineal al duplicar n sin aumentar d. La prueba adicional con n fijo y $d = 3, 6, 9$ permite observar el incremento del trabajo al crecer la cantidad de cifras procesadas.
Medición del equipo. Para entrada aleatoria y $n = 1\,024\,000$ la mediana fue 33,693 ms y la última razón 1,97. Con valores duplicados entre 0 y 10 se obtuvo 11,158 ms, lo que es compatible con realizar menos pasadas decimales (hasta dos dígitos en vez de seis).

4.5. Prueba especial de Radix y cierre de resultados

Dígitos (d)	Mediana (ms)
3	1.617
6	3.060
9	4.710

Tabla 15: Prueba especial de Radix Sort con $n = 100\,000$ y base decimal.

En la medición especial del equipo, a $n = 100\,000$ fijo, Radix registró 1,617 ms con tres dígitos, 3,060 ms con seis y 4,710 ms con nueve. El incremento es compatible con la necesidad de procesar más posiciones decimales cuando aumenta d, sin que cambie n.
Como limitaciones adicionales, las ejecuciones cortas pueden verse afectadas por la JVM y por otras tareas del sistema operativo; además, en los tres casos de desbordamiento de pila no existe una mediana válida para calcular razones posteriores. El archivo CSV suministrado conserva los tiempos individuales para comprobar los resultados presentados; la interpretación se limita a los experimentos ejecutados.

Referencias

[1] Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica. Tarea extraclase #2: Benchmark de algoritmos de ordenamiento, 2026. Algoritmos y Estructuras de Datos I (CE 1103), II Semestre 2026, valor 6 %.

[2] OpenAI. Chatgpt: conversación compartida sobre la elaboración de prompts para algoritmos de ordenamiento. `https://chatgpt.com/share/6af3add2-c914-83e8-8876-5983fff56c26`, 2026. Consultada el 10 de noviembre de 2026.

Anexos

Código fuente

El código fuente de los algoritmos de ordenamiento y de los programas de verificación utilizados en este trabajo se encuentra disponible para su consulta en el siguiente repositorio de GitHub: `https://github.com/Tokitou14/Tarea-II.git`